Readmission Alerting

Tensium Samples

Pack identification

Category

CPU ML engineering

Pack / category name

Readmission Alerting

Why current models fail, and the capability this task builds

Two of four runs pass. Both failures correct the headline metric yet stop one level short target encoding that leaks across folds (gap ~0.04 versus ~0.005 for passes) rather than properly nested, out-of-fold encoding. The capability isolated is subtle leakage detection: recognizing that a plausibly-correct evaluation still quietly contaminates itself. This is the exact defect that inflates offline metrics in production ML, so a model that learns to catch it gains the ability to audit its own evaluations.

1. Description

Description

A hands-on CPU machine-learning engineering task built around an evaluation figure that nobody on the care team can reconcile with what they see. The agent works out why the published number is wrong, repairs the pipeline, deploys it through a gated release service, recovers from two staged-state faults, and drives the release to submission. The figure is wrong for four independent reasons, and fixing any three of them still leaves it outside tolerance.

2. Task type & unit type

Task type

Hands-on long-horizon: inspect -> reproduce -> edit -> deploy -> validate -> diagnose -> recover -> validate -> diagnose -> recover -> validate -> promote -> submit.

Unit type

One isolated Harbor task instance containing a real encounter table, an editable pandas/sklearn pipeline, a gated operations service, a verifier-side reference and a deterministic grader.

3. Verification method

Verification method

Binary reward by behaviour. The deployed source is executed in an unprivileged subprocess and the figure it publishes is compared by value against a reference the verifier computes with its own code. The comparison is repeated on a re-balanced copy of the cohort, so a figure that is computed follows the data and a figure that is emitted does not. Full reward additionally requires the ordered gate walk and both forced recoveries. Four structurally different correct implementations all score 1.0, which is what shows grading is by value rather than by source shape.

4. Technical domains / adjacent benchmarks

Technical domains

Classical ML engineering; pandas cohort construction; point-in-time observability of outcomes; target encoding and encoding leakage; ranking metrics for rare events; deterministic offline evaluation.

Adjacent benchmarks

Terminal-Bench-like coding environment with tau-bench-like gated state transitions and hidden deterministic value verification.

5. Human experts used

Human experts

The task was authored and calibrated. Review and QA feedback was supplied by the Tensium task-review team. No additional credential claims are made.

6. Raw base data

Raw base data

UCI Machine Learning Repository "Diabetes 130-US hospitals for years 1999-2008" (real inpatient encounter records). The agent-visible slice is patients with more than one admission: 16,845 encounters across 6,000 patients, 19.6% readmitted within 30 days. The discriminating signal comes from genuine properties of the records. A patient's final encounter in the extract has no observable follow-up window, and the diagnosis-code triple is close to unique per encounter at 12,886 distinct values, so a rate encoding fitted without nesting hands a training row its own outcome back as a feature.

7. What the world looks like

What the world looks like

The agent starts as an unprivileged Ubuntu user in /workspace/target with editable Python sources, the real encounter table, config, a data card, smoke tests and an evaluation entry point. Network access is disabled. A change-control service governs observations, deployment, rate-limited one-bit validation, diagnostics, metered recovery, promotion and submission. The grader, the reference and the release fingerprint remain outside the writable workspace.

8. Time per task

Time per task

Approximately 8–10 authoring and QA hours per task.

9. QA process

QA process

QA runs through a systematic, automated pipeline rather than an ad-hoc pass. Deterministic controls mechanically screen the task for the failure modes that matter reward-hacking and forgeable graders, answer leakage and reachable oracles, in-process grading, and false difficulty (for example, a "failure" that was only budget truncation). Each control is two-condition validated: it must fire on a planted defect and stay silent on a clean task. An adversarial control battery then executes cheat submissions against the real grader and requires every one to score zero, while a gold solution and a structurally different correct solution both score 1.0. An independent review pass grounds every finding in the actual task files, and difficulty is calibrated by running a frontier model at a fixed sampling standard inside the real isolated environment, with guards that separate genuine failures from budget/truncation artifacts.

Completed TASK-QA-CHECKLIST coverage: 21 offline control scenarios, all green. Six positive controls score 1.0 and fifteen negative and cheat controls score 0.0, each flooring for its own stated reason rather than incidentally. Deterministic repeats; grader isolation; unprivileged subprocess execution; verifier-directory lock; package leak guard; secret scan; and a review of every real model trace. Four of the positive controls are structurally different correct implementations, two of them produced by models rather than the author.

10. Difficulty calibration

SOTA model & pass@1

Claude Opus 5: 4 independent runs with an 80-step maximum; 2/4 strict passes (50%). The two passing runs met both grading conditions comfortably: condition A gaps of 0.0016 and 0.0049 against a 0.012 tolerance, and condition B of 0.0051 and 0.0157 against a 0.05 tolerance. The two failing runs each left the rate encoding un-nested and published an unsound figure (condition A gaps of 0.0383 and 0.0420; reported average precision near 0.295 against a 0.3367 reference), landing near the same defect figure from separate trajectories. No infrastructure failures.

Rollout records

Local Claude Code records: readmission-alerting_opus5_4traces.zip. Frozen task package SHA-256: 20190b97e186e43902dd0e51e6839f2594936305076c40463f84e066bdf9c891

11. Grading-design table

Component / gate / axis	What it measures	Weight / cap	How enforced
Env-owned terminal	Published figure against a reference the verifier computes itself	hard requirement	Value comparison within 0.012; deployed source run in an isolated subprocess
Responsiveness	Whether the figure is computed or emitted	hard requirement	Second measurement on a re-balanced cohort, tolerance 0.05, measured separately
Ordered gate walk	Required investigation, deploy, recovery and release sequence	hard requirement	Action ledger replay against the required order
Forced recovery	Two diagnosed faults, each cleared with a matching family	hard requirement	Dirty validation -> diagnostics -> matching recovery, twice
Isolation / anti-cheat	Candidate cannot read or forge verifier state	hard->0.0	Unprivileged worker, result file, verifier directory chowned root 0700
Final reward	All behavioural and trajectory requirements	binary 0.0/1.0	Boolean conjunction; partial progress never compensates

12. How the tasks were created

How created

The task was built from real properties of the UCI encounter records rather than an injected fault. Author-side measurement established which candidate defects move the graded figure beyond a workable tolerance and which do not; several plausible ideas were discarded because they measured under 0.02 and would not have discriminated. Four requirements survived: the analysis cohort must be restricted to index admissions, the published figure must be the metric the config commits to, the rate encoding must be fitted inside each fold, and that encoding must be nested so no row's own outcome reaches its own features. Each is stated in the config or the data card and none is given as a recipe. The cohort rule is the subtlest: the shipped code drops each patient's last row by file order, the export is unordered, and encounter_id is what carries recency, so the row count looks right while

a third of the cohort is wrong. Tolerance was set from measurement rather than judgement: six correct implementations span 0.0000 to 0.0051 and the nearest defect sits at 0.039. Two staged artifacts are restored at every deploy, so a corrected pipeline has no effect on the published figure until both are diagnosed and cleared. The deciding signal is forced through operations, validation results redirect diagnosis, and elective depth earns nothing.

---

13. QA checklist

☒ Two-condition proof: positive control (oracle scores 1.0) AND negative controls (wrong work scores 0.0, for the intended reason).

☒ For hands-on tasks: a SECOND, structurally different correct solution also scores ~1.0 (grader accepts by behaviour, not gold's shape).

☒ Grader isolation: grader, answers, and reference solution live outside the agent's /workspace/target/ (in tests/ and solution/).

☒ For hands-on tasks: agent code runs in the verifier as a subprocess with a timeout - never imported into the grader.

☒ No answer / solution leakage: nothing in environment/workspace/ reveals the answer; reference deliverable absent; Dockerfile build guard enforces it.

☒ Deducibility: everything needed to solve is in the workspace; task is actually solvable; no invisible coin-flips.

☒ Real data, not synthetic: built on real OSS repos / public datasets / real experiment artifacts.

☒ Long-horizon by design: the staged artifacts force the operational loop; results redirect the next step; trajectory depth is reported but never a scoring term.

☒ Build hygiene: zip built in a temp/staging dir; solution/ is excluded from the agent's runtime workspace (present only for reviewer verification).

☒ Scrubbed: no secrets/keys, no platform job-ids or URLs, no internal project names or process jargon; no reproduced customer sample content.

☒ Realistic and professionally relevant: the task models a production evaluation that reports a figure a constant baseline would beat.

☒ Clearly specified and internally consistent: the agent instructions, data card, config, visible tests, gated operations sequence and deterministic grader agree on the required workflow.

☒ Reproducible and technically functional: pinned dependencies, fixed seeds, a frozen release fingerprint and containerized verification.

☒ Distinct and non-duplicative: the package contains one self-contained task and no duplicate task variants.

☒ Licensed and non-sensitive source material: Diabetes 130-US hospitals is a public UCI dataset; no private material is added.

☒ Delivery format: the package follows the Harbor task structure with instruction, task metadata, environment, tests and solution directories.

14. Package overview & contents

This submission covers one task package in detail. The package is self-contained and includes the agent instructions, editable workspace, container environment, verifier-side assets, deterministic grader and reference solution.

Field	Submission detail
Selected package	readmission-alerting.zip
Task count	1 self-contained Harbor task
Task	cpu-ml-engineering/readmission-alerting
Task type	CPU ML engineering; hands-on long-horizon investigation, repair, deployment and recovery
Unit / deliverable	One isolated task instance; corrected deployed source plus clean validation, promotion and submission
Verification	Binary behavioural grading against a verifier-computed reference under two conditions, plus an ordered gate walk and two forced recoveries
Raw base data	UCI Diabetes 130-US hospitals encounter records

15. Per-task deep-dive

Task	What the task is about	Deliverable
Readmission Alerting	Work out why the published evaluation figure cannot be reconciled with what the care team sees, repair the cohort, the metric and the feature encoding, then complete the gated deploy, validation, recovery and release sequence.	A corrected deployed pipeline whose published figure matches the verifier's reference under both grading conditions, followed by clean validation, promotion and successful submission.

16. Example prompt

The readmission alerting pipeline in /workspace/target feeds the care team's follow-up list: every discharge gets a risk score, the list is ranked, and the team works down from the top until the afternoon runs out. The figure we publish for it is 0.95, and nobody can square that with what the team sees - six weeks in, the hit rate feels like noise, and one of the nurses asked whether they'd do just as well calling people at random. Two people have read the model code and found nothing wrong with the model, which points at the number itself rather than the model.

Work in /workspace/target, no internet, everything installed. You can edit anything under src/ (load.py, features.py, splitting.py, evaluate.py), but keep the public API of src/evaluate.py intact - run_evaluation(df, cfg), fit_final(df, cfg), and predict_scores(model, df, cfg) are imported by a downstream harness, so names and signatures stay. Don't touch data/encounters.csv. Read config.yaml and data/DATA_CARD.md for the cohort definition and how the figure is used, run scripts/run_eval.py and the smoke tests, and use inspect, profile, and sample_records to gather evidence before changing anything.

Changes go out through the ops service, not by editing and walking away - drive it with python3 env_cli.py <op> in order: reproduce -> deploy -> validate, and if validate comes back dirty, diagnostics.read then recovery.apply --family <the family the evidence points to> -> validate again until clean -> promote -> submit. validate returns a single clean/dirty bit and is rate-limited, so use it to confirm something you've already worked out rather than to search; recovery.apply is refused until you've run diagnostics.read, and a wrong family is rejected and metered. Grading uses the terminal the environment recomputes at submit from what you deployed - editing results/eval_report.json achieves nothing. You're done when the published figure is one the care team could actually act on, driven through validation and recovery to submit.

17. Environment & execution

Requirement	Specification
Runtime	Python 3.12 slim container; unprivileged ubuntu agent workspace
Dependencies	numpy 2.2.6; pandas 2.3.3; scikit-learn 1.7.2; PyYAML 6.0.3
Compute	2 CPUs; 4,096 MB memory; 10,240 MB storage; 0 GPUs
Time limits	Agent: 3,600 seconds; verifier: 2,700 seconds; environment build: 3,600 seconds
Visible tests	python3 -m unittest discover -s tests
Evaluation	python3 scripts/run_eval.py
Operations	python3 env_cli.py OP, followed by the required ordered gate sequence
Verification	tests/test.sh executes python3 /tests/grade.py
Network policy	Internet access disabled

18. Human, AI & automation roles

Contributor	Role in design and production
Human experts	Selected and measured the public encounter data; established which defects discriminate and which do not; authored the environment, reference and

Contributor	Role in design and production
	controls; performed security hardening; reviewed every model trace and approved the frozen task.
AI systems	Used for calibration rollouts and behaviour testing. Four Claude Opus 5 runs measured difficulty. Two model-produced implementations were kept as permanent controls, which is what tests that grading is not tied to one source shape.
Automation	Container builds, leak guards, visible smoke tests, isolated subprocess execution, release-state fingerprinting, action-ledger checks, staged-fault injection, deterministic repeats and twenty-one control scenarios enforce reproducibility and grading integrity.

19. Supporting artifacts

Artifact	Contents
readmission-alerting.zip	Complete Harbor task package: instruction, task metadata, environment, workspace, tests, verifier assets and reference solution.
readmission-alerting_opus5_4traces.zip	Four independent Claude Opus 5 calibration traces with grader output, rewards, action logs, tool counts and the source each run deployed.
Section 13 of this document	Completed quality-control checklist, including positive and negative controls, isolation, leakage, reproducibility, licensing and delivery-format checks.

20. Acknowledgement

The vendor confirms that the task has been reviewed and is realistic, technically substantive, clearly specified and appropriately graded. It reflects a genuine professional workflow and is not a low-effort or artificial task. The creation and calibration statements above are accurate, and the QA checklist has been completed.